REAL-TIME TRAFFIC ANALYTICS

Project Planning & Management

Submission 1 • Planning, responsibilities, schedule, and project control

1. Project Overview

Real-Time Traffic Analytics is a prototype that receives simulated live traffic events, processes them as they arrive, identifies congestion and abnormal traffic conditions, stores raw and processed records, and displays key indicators on an interactive dashboard. If sufficient usable data and time are available, the team will evaluate a basic model that estimates whether congestion is likely to occur shortly ahead.

2. Project Objectives

- Generate simulated traffic events containing vehicle count, average speed, timestamp, and location.
- Ingest the event stream through Azure Event Hubs and process it using Azure Stream Analytics.
- Calculate traffic indicators and detect congestion or anomalies using transparent, configurable rules.
- Store raw, cleaned, and analytics-ready records using a Bronze/Silver/Gold organization in Azure Data Lake Storage.
- Present traffic volume, average speed, congestion status, trends, and alerts in Power BI.
- Send notifications through Azure Logic Apps when configured thresholds or anomaly conditions are met.
- Evaluate a basic congestion-prediction model only if data quality and available project time permit.

3. Scope and Boundaries

Included in the minimum working prototype:

- Python-based traffic simulator producing JSON events.
- Streaming ingestion, rule-based processing, storage, dashboard visualization, and test notifications.
- Documented test scenarios covering normal traffic, congestion, sudden speed changes, unusual vehicle counts, and invalid or missing fields.
- End-to-end integration test and final report/presentation.

Optional extensions: a basic congestion-prediction model using Azure Databricks, and Azure Data Factory only if scheduled/batch orchestration becomes necessary. These are not required for the minimum working prototype.

4. Team Structure and Responsibilities

Member	Assigned Task	Main Responsibilities	Expected Output
Menna Muhammed	Task 1 — Python Data Simulation	Design JSON schema; generate realistic events; include vehicle count, speed, timestamp, and location; test consistency and scenarios.	Python simulator, JSON schema, sample dataset, test scenarios
Habiba Aymam	Task 2 — Azure Event Hubs	Configure Event Hubs; connect the Python producer; verify ingestion; test formatting and stream delivery; document setup.	Working ingestion path and setup notes
Bahaa Mohamed	Task 3 — Azure Stream Analytics	Create inputs/outputs and streaming queries; calculate indicators; define congestion/anomaly rules; evaluate simple prediction if feasible.	Stream-processing job, rules, query documentation, test results
Mahmoud Gamal	Task 4 — Data Lake Storage	Configure storage; define Bronze/Silver/Gold structure; store raw and processed records; document schemas and access.	Storage structure, sample records, data-layer documentation
Eman Samaha	Task 5 — Power BI Dashboard	Build visuals for volume, average speed, congestion status, and trends; connect analytics-ready output; validate metrics.	Interactive dashboard and presentation screenshots
Batoul Elnahas	Task 6 — Alerts & Integration	Coordinate integration; track blockers and access; configure Logic Apps notifications; test end-to-end flow; consolidate report and presentation.	Alert workflow, integration checklist, final documentation

5. Four-Week Schedule

Week	Focus	Key Activities	Deliverables
Week 1	Requirements, design, and data generation	Agree scope and success criteria; draft architecture; define JSON schema; build initial simulator; plan ingestion, storage, indicators, dashboard, and alerts.	Approved scope, architecture diagram, task checklist, JSON schema, sample data, draft configurations
Week 2	Streaming ingestion and core processing	Complete continuous simulator; connect producer to Event Hubs; create Stream Analytics job and initial queries; create Data Lake structure; prepare dashboard draft.	Working Python-to-Event-Hubs stream, initial traffic indicators, storage structure and sample records, dashboard wireframe
Week 3	Congestion detection, storage, and dashboard	Test normal and abnormal scenarios; implement congestion/anomaly rules; assess prediction feasibility; organize Bronze/Silver/Gold layers; connect dashboard; configure alerts.	Tested rules, layered data storage, interactive dashboard, first alert workflow
Week 4	Integration, testing, and final presentation	Run end-to-end tests; validate event quality, processing outputs, stored data, dashboard metrics, and notifications; fix issues; prepare final report and demo.	Integrated prototype, test results, final dashboard, tested notifications, report, presentation, individual summaries

6. Project Monitoring and Communication

- Maintain a shared task checklist with an owner, status, due date, and blocker for each deliverable.
- Agree on the JSON event schema and field names before connecting components.
- Each member should document setup steps, assumptions, test evidence, and known limitations for their component.
- Integrate components incrementally rather than waiting until the final week.
- Raise access, credentials, data-format, or connectivity issues as soon as they appear.

7. Success Criteria

- The simulator continuously emits valid JSON traffic events.
- Events reach Event Hubs and are processed without systematic schema errors.
- Configured rules flag representative congestion and abnormal scenarios correctly in the test dataset.
- Raw and processed records are stored in the intended data layers.
- The dashboard displays values consistent with the processed data.
- At least one test notification is delivered through the configured alert workflow.
- The team can demonstrate the end-to-end pipeline and explain its limitations.

8. Risks and Mitigation

Risk	Impact	Mitigation
Azure access, permissions, or cost constraints	Delays in cloud integration	Verify accounts and permissions early; test each service independently; use the smallest feasible prototype configuration.
Inconsistent or invalid event fields	Failed queries or misleading metrics	Freeze the JSON schema early; validate events; test missing and malformed values.
Insufficient data for ML prediction	Prediction feature may not be reliable	Keep prediction optional; establish a baseline and report limitations rather than claiming unsupported accuracy.
Dashboard refresh/integration limitations	Metrics may not appear instantly	Validate the selected connection and refresh behavior early; describe the dashboard as live or frequently refreshed according to actual behavior.
Late integration	End-to-end demo may fail	Run staged integration tests in Weeks 2 and 3 and reserve Week 4 for fixes and verification.

9. Deliverable Note

This plan follows the four-week implementation schedule and the six responsibilities defined in the project proposal. Actual dates should be confirmed with the course instructor and team.

REAL-TIME TRAFFIC ANALYTICS

Literature Review

Submission 2 • Background research and relevance to the proposed prototype

1. Introduction

Traffic congestion is a major challenge for urban transportation systems. Traffic analytics systems seek to turn observations such as vehicle counts, speeds, and timestamps into timely indicators that can help users understand current road conditions. The proposed project focuses on a small, demonstrable pipeline: simulated traffic events are streamed, processed, stored, visualized, and used to trigger notifications.

2. Real-Time Traffic Analytics and Stream Processing

Streaming analytics differs from a workflow that waits until a large batch of data has accumulated. Events can be ingested and processed as they arrive, while time windows support rolling summaries such as average speed or vehicle count. Microsoft's Azure Architecture Center describes Event Hubs as a stream-ingestion service and Azure Stream Analytics as a managed engine for SQL-like stream processing, including filtering and aggregation over time windows [1]. Microsoft's reference architecture also describes a common pattern in which simulated or device-generated events are sent to Event Hubs, processed by Stream Analytics, and routed to storage or analytics consumers [2].

This architecture is relevant to the project because the Python simulator can act as a repeatable event source, Event Hubs can provide the ingestion layer, and Stream Analytics can calculate traffic indicators and apply explicit rules. The prototype will demonstrate the pipeline with simulated data; it will not claim to be connected to live road sensors unless such a data source is added.

3. Congestion Detection and Anomaly Identification

Congestion detection can be implemented initially through transparent rules based on traffic indicators. For example, a configurable combination of lower average speed and higher vehicle count may flag a congestion condition. Anomaly rules can flag unusual vehicle counts, abrupt speed changes, or invalid event values. The exact thresholds must be selected and tested by the team; the proposal does not specify final numerical thresholds.

A rule-based baseline is appropriate for a student prototype because it is explainable and can be validated with deliberately generated test cases. It also separates two concepts: detecting a condition that is present in the incoming data and predicting a condition that may occur in the near future. The project should report these as separate capabilities.

4. Traffic Prediction and Machine Learning

Research reviews identify traffic-flow forecasting and congestion prediction as active areas within intelligent transportation systems. Akhtar and colleagues' review discusses AI methods for traffic congestion prediction and notes the importance of traffic parameters and short-term forecasting [3]. Sayed, Abdel-Hamid, and Hefny review AI-based traffic-flow prediction methods and situate them within intelligent transportation systems [4]. Kumar and Raubal survey deep-learning applications for congestion detection, prediction, and alleviation [5].

These reviews support the general idea that traffic observations can be used to estimate future traffic conditions, but they do not establish that any particular model will work well on this project's simulated data. For this reason, the proposal treats prediction as an optional advanced component. If enough representative and consistently labelled data can be generated, the team may train and evaluate a basic model that predicts whether congestion is likely in a short future interval. If the data are insufficient, the team should deliver and evaluate the rule-based detection system and clearly document prediction as future work.

5. Data Storage and Visualization

A layered data organization can separate original incoming records from cleaned records and analytics-ready outputs. In this project, the proposed Bronze layer will retain raw events, the Silver layer will hold validated or cleaned records, and the Gold layer will contain prepared indicators suitable for reporting. Power BI will present traffic volume, average speed, congestion status, trends, and processed outputs. The exact latency of dashboard updates depends on the implemented data connection and refresh configuration, so the report should describe observed behavior rather than assume every visual is instantaneously updated.

6. Alerts and Operational Use

A traffic dashboard makes conditions visible, while notifications can draw attention to selected conditions. The proposal uses Azure Logic Apps to route notifications when configured congestion thresholds or anomaly conditions are met. Microsoft's documentation on stream-processing patterns describes event-driven alerting as a common use of stream processing, with processing outputs routed to downstream services [6]. For the prototype, alerts should be tested with known trigger and non-trigger scenarios to avoid presenting unverified notifications as reliable operational warnings.

7. Research Gap and Project Contribution

The project is not intended to introduce a new traffic-prediction algorithm. Its contribution is an integrated educational prototype that demonstrates the movement of traffic events through ingestion, stream processing, storage, visualization, and notification. It also distinguishes rule-based current-condition detection from optional future-condition prediction and documents the limitations of simulated data.

8. Conclusion

The reviewed literature and technical architecture guidance support combining streaming ingestion, windowed processing, traffic indicators, storage, dashboards, and event-triggered alerts. The project will implement a manageable baseline using simulated JSON events and transparent rules. A machine-learning prediction component will only be included if the team can produce sufficient data and evaluate it appropriately.

References

[1] Microsoft. “Choose a Stream Processing Technology.” Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/data-guide/technology-choices/stream-processing

[2] Microsoft. “Stream processing with Azure Stream Analytics.” Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/reference-architectures/data/stream-processing-stream-analytics

[3] M. Akhtar et al. (2021). “A Review of Traffic Congestion Prediction Using Artificial Intelligence.” Journal of Advanced Transportation. https://doi.org/10.1155/2021/8878011

[4] S. A. Sayed, Y. Abdel-Hamid, and H. A. Hefny (2023). “Artificial intelligence-based traffic flow prediction: a comprehensive review.” Journal of Electrical Systems and Information Technology, 10, 13. https://doi.org/10.1186/s43067-023-00081-6

[5] N. Kumar and M. Raubal (2021). “Applications of deep learning in congestion detection, prediction and alleviation: A survey.” Transportation Research Part C: Emerging Technologies, 133, 103432. https://doi.org/10.1016/j.trc.2021.103432

[6] Microsoft. “Azure Stream Analytics solution patterns.” Microsoft Azure documentation. https://github.com/MicrosoftDocs/azure-docs/blob/main/articles/stream-analytics/stream-analytics-solution-patterns.md

REAL-TIME TRAFFIC ANALYTICS

Requirements Gathering

Submission 3 • Functional, non-functional, data, and interface requirements

1. Purpose

This document defines the requirements for the Real-Time Traffic Analytics prototype described in the project proposal. The prototype uses simulated JSON traffic events and an Azure-based streaming architecture to detect current congestion and anomalies, store data, display indicators, and issue configured notifications.

2. Stakeholders and Users

Stakeholder/User	Interest or Need
Project team	Build, integrate, test, and document the prototype.
Dashboard viewer / traffic analyst (demonstration user)	Understand current simulated traffic conditions, trends, congestion indicators, and alerts.
Project evaluator / instructor	Review the architecture, requirements, implementation, test evidence, and limitations.

3. System Context and Data Flow

– Python traffic simulator generates JSON events.

– Azure Event Hubs ingests the continuous event stream.

– Azure Stream Analytics processes events, calculates indicators, and applies congestion/anomaly rules.

– Azure Data Lake Storage retains raw, cleaned, and analytics-ready data in Bronze, Silver, and Gold layers.

– Power BI displays the available traffic metrics and trends.

– Azure Logic Apps routes notifications when configured rules are triggered.

– Optional: a basic ML model may estimate near-future congestion if enough suitable data is available.

4. Functional Requirements

ID	Requirement	Priority	Acceptance / Verification
FR-01	The system shall generate simulated traffic events using Python.	Must	Run the simulator and observe a continuous sequence of events.
FR-02	Each event shall include vehicle count, speed, timestamp, and location.	Must	Inspect sample JSON events and confirm required fields and types.
FR-03	The simulator shall export events in JSON format.	Must	Validate output syntax and schema consistency.
FR-04	Azure Event Hubs shall receive events from the Python producer.	Must	Send test events and verify they arrive at the ingestion layer.
FR-05	Azure Stream Analytics shall process incoming events using time-based queries or windows where appropriate.	Must	Run the job and inspect processed output.

ID	Requirement	Priority	Acceptance / Verification
FR-06	The processing layer shall calculate traffic indicators such as vehicle count and average speed.	Must	Compare output values with manually calculated test values.
FR-07	The system shall flag congestion using configurable traffic rules.	Must	Run a known congestion scenario and verify the expected flag.
FR-08	The system shall flag configured abnormal conditions, such as abrupt speed changes or unusual vehicle counts.	Must	Run anomaly scenarios and verify expected flags.
FR-09	The system shall retain raw and processed records in Azure Data Lake Storage.	Must	Confirm sample records exist in the designated storage layers.
FR-10	Stored data shall be organized into Bronze, Silver, and Gold layers.	Must	Inspect folders/containers and confirm documented layer purpose.
FR-11	Power BI shall display traffic volume, average speed, congestion status, and trends from available processed data.	Must	Compare dashboard values against source outputs.
FR-12	Azure Logic Apps shall send a notification when a configured threshold or anomaly condition is met.	Must	Trigger a test condition and verify the notification is received.
FR-13	The team shall run an end-to-end test across generation, ingestion, processing, storage, dashboard, and alerts.	Must	Document a successful run, observed failures, and corrective actions.
FR-14	The team may train and evaluate a basic model to predict near-future congestion if sufficient data and time are available.	Optional	If implemented, document target label, prediction horizon, train/test split, evaluation metrics, and limitations.

5. Non-Functional Requirements

ID	Requirement	Verification / Notes
NFR-01	Timeliness: the pipeline should process events quickly enough to demonstrate near-real-time behavior.	Measure or record observed delay; do not claim a latency target unless the team defines and tests one.
NFR-02	Data quality: events should follow the agreed schema and malformed/missing fields should be identified.	Test valid, missing-field, and invalid-value examples.
NFR-03	Reliability: the team should verify ingestion and processing across repeated test runs.	Repeat test scenarios and record failures or dropped events where measurable.
NFR-04	Usability: dashboard indicators should be clearly labelled and understandable.	Review labels, units, status definitions, and visual consistency.
NFR-05	Maintainability: queries, rules, schema, and configuration steps should be documented.	Check that another team member can follow the setup notes.
NFR-06	Security: cloud credentials and access permissions should be handled responsibly.	Do not place secrets in source code or the report; use appropriate access controls.

ID	Requirement	Verification / Notes
NFR-07	Cost awareness: use only the Azure resources needed for the prototype and monitor resource use.	Document services used and shut down or remove resources when no longer needed.
NFR-08	Reproducibility: key test scenarios and representative sample data should be repeatable.	Retain test inputs, expected outcomes, and test records.

6. Data Requirements

The proposal names vehicle count, speed, timestamp, and location as event fields. Additional derived fields above are proposed for processing and reporting; the team should confirm final field names and units before implementation.

Field	Description	Requirement / Validation
timestamp	Time at which the simulated traffic event is generated or observed.	Required; use a consistent documented format and timezone convention.
location	Identifier or label for the road segment/location represented by the event.	Required; values should follow an agreed naming convention.
vehicle_count	Number of vehicles counted for the represented location/time window.	Required; should be numeric and non-negative.
average_speed	Average vehicle speed for the represented location/time window.	Required; numeric, non-negative, with units documented.
congestion_status	Rule-derived status assigned during processing, if applicable.	Derived field; rule and threshold definitions must be documented.
anomaly_flag	Indicates whether a configured anomaly condition was detected.	Derived field; record which rule triggered the flag where feasible.

7. Interface Requirements

- The Power BI dashboard shall use clear titles, labels, and documented units for traffic volume and average speed.
- The dashboard shall distinguish the current rule-based congestion status from any optional machine-learning prediction.
- The dashboard shall show trends over time using the timestamps available in processed records.
- Alert messages shall identify the triggered condition and, where available, the relevant location and timestamp.
- The final demonstration shall explain the meaning of each indicator and disclose whether dashboard updates are live or periodically refreshed.

7. Business Rules and Assumptions

- The initial data source is a Python simulator, not a live traffic sensor feed.
- Congestion thresholds and anomaly conditions must be defined, documented, and tested by the team; no fixed numerical thresholds are specified in the proposal.
- Rule-based detection of current conditions is part of the minimum prototype.
- Machine-learning prediction is optional and must not be described as completed unless implemented and evaluated.
- Dashboard refresh behavior depends on the actual Power BI connection and configuration.
- Azure Data Factory is optional and should only be added if scheduled/batch orchestration is needed.
- Azure Databricks is an optional route for training/evaluating a basic prediction model.

8. Out of Scope

- Guaranteed prediction accuracy or production-grade traffic forecasting.
- Direct integration with real-world road cameras, roadside sensors, or municipal traffic systems unless separately approved and implemented.
- Automated traffic-light control or real-world intervention.
- A production service-level agreement or a guaranteed end-to-end latency.

- Mandatory use of Azure Databricks or Azure Data Factory.

9. Acceptance Checklist

- ■ JSON event schema is documented and sample events validate.
- ■ Python producer sends events to Event Hubs.
- ■ Stream Analytics produces indicators and congestion/anomaly outputs.
- ■ Raw and processed records are stored in the intended Data Lake layers.
- ■ Power BI displays metrics that match the processed output.
- ■ At least one Logic Apps notification scenario is verified.
- ■ End-to-end test results and known limitations are included in the final report.